Reflection

1. Problems With AI Responses

During development of MiniDash, I worked with ChatGPT to assist with writing code, testing and adding new functions. While AI assistance was useful, some of the suggestions generated by ChatGPT needed corrections. Two examples of flaws in ChatGPT's response included a confusion over the status PLACED for scheduled orders and an additional flag to indicate whether the kitchen had received the order. Because both scheduled orders and regular orders could display the status PLACED, it would appear that scheduled orders had been placed although they had not yet reached a stage where they could be prepared. I identified this flaw while examining the design of the order status workflow. I requested that ChatGPT generate a second status, SCHEDULED, which would allow future orders to remain at SCHEDULED until thirty minutes prior to their scheduled time for delivery, at which point they would automatically transition to PLACED. Normal or "regular" orders would continue to start with PLACED. The revised order workflow was much clearer and easier to test.

A second error in ChatGPT's initial response occurred due to ChatGPT's reliance upon the system clock for scheduling. In addition to making it difficult to write unit tests to validate the timing of the scheduling rules (for example, the forty-five minute minimum, seven day maximum and thirty minute release cut off), relying on the system clock made it difficult to simulate exact times during testing. Therefore, I requested that ChatGPT develop two classes, Clock and FakeClock. FakeClock allowed me to specify a time for testing purposes instead of having to rely upon the actual time. This greatly reduced the amount of time spent testing and improved my ability to consistently run tests against the various scheduling restrictions.

2. Areas Where AI Provided Greatest Assistance And Least Assistance

I found that the greatest assistance from AI was in providing methods for automated testing of MiniDash. ChatGPT demonstrated how to utilize pytest and FastAPI's TestClient to enable me to test my API without having to manually start a server. Additionally, ChatGPT assisted me in creating a separate temporary SQLite database that enabled me to isolate testing for customer orders from normal operation. Overall, this greatly simplified the testing experience and enhanced my understanding of how to verify whether my functionality was properly implemented.

Conversely, I experienced little assistance from AI when ChatGPT suggested overly complex solutions. A good example of this was ChatGPT's recommendation for an extra flag for scheduled orders rather than utilizing a different status for those orders. After reviewing ChatGPT's suggestion, I clearly explained why a more straightforward solution was preferred. This experience illustrated that AI does not always offer optimal solutions for smaller projects

such as MiniDash and that it is still necessary for developers to review suggestions offered by AI systems.

3. Changes To Prompts Used By Me During Development Of Project

In early stages of development of MiniDash, my requests to ChatGPT were more open-ended. Initially, I simply asked ChatGPT to describe possible ways for organizing MiniDash and how to implement the backend using FastAPI and SQLite. As development progressed, however, I began providing increasingly detailed directions regarding what I desired from ChatGPT's recommendations and what aspects of the implementation I wished to maintain unchanged.

When implementing scheduled delivery capabilities for customers, for example, I specified that customer orders must be scheduled at least 45 minutes but no greater than 7 days in advance of delivery. Furthermore, I instructed ChatGPT to utilize FakeClock to simulate the precise delivery timing constraints. Finally, later in development, I instructed ChatGPT to review the scheduling logic, add any missing tests and only modify the backend if there actually existed a legitimate issue with it. Also, I specifically told ChatGPT that regular orders must function exactly as they had previously. Providing ChatGPT with more detail regarding the requirements of each request resulted in ChatGPT generating more accurate and relevant responses.

4. A Decision That I Did Not Delegate

A decision I made independently was to make sure that MiniDash was structured as simply and clearly as possible so I could develop the project on my own. The modular monolith architecture was suggested by ChatGPT; however, since MiniDash is a small, single-person academic project, I chose to use this design to organize the API, database, scheduler, and frontend into separate files without making the application overly complicated. This design allowed me to understand, maintain, and explain the application more easily during my presentation. Although ChatGPT suggested the architecture, I ultimately decided that it was the best approach for my project.

5. Limitations of My Data/Evidence

Although MiniDash has automated tests that check its API and scheduling features, there are still some areas that have not been fully tested. One example is how MiniDash performs when running continuously for a long period of time. As previously mentioned, FakeClock helps simulate when orders should be released, but it does not fully verify whether the scheduler will continue processing orders correctly over an extended period. Another area I have not fully tested is what happens when multiple customers place orders at the same time or when the server restarts while scheduled orders are waiting to be released. If I had additional time, I would test multiple orders being placed simultaneously, restart the server to check whether scheduled orders are still processed correctly, and perform more thorough testing of the entire frontend.